Listing 5.4 Naïve solution, areDisjoint

```
function areDisjoint(elem1, elem2)
                                                #1
  p1 = this.findPartition(elem1)
                                    #2
  p2 =
this.findPartition(elem2)
         #3
  return p1 !=
p2
#4
```

#1 Takes two elements and returns `true` iff the elements are valid but don't belong to the same partition, `false` iff the elements are valid but do belong to the same partition. Notice that if either element is `null` or hasn't been added to this container, then this method will throw (because in turn `findPartition` will throw an error).
#2 Retrieve the disjoint set to which `elem1` belongs to. If the argument is invalid or not found, this call will throw.
#3 Repeat the same operation for `elem2`.
#4 Compare the two sets, and check if they are the same, and hence if the elements belong to the same partition.

We just need to reuse `findPartition`, call it for both elements, and check both calls return the same partition. Note that, by reusing `findPartition`, we can make sure that the implementation of `areDisjoint` won't need to change, no matter how our elements are stored nor `findPartition` is implemented (as long as its interface remains the same, and partitions can be compared with the inequality operator).